

MAE 3780 — Final Report

Mechatronics Robot Project

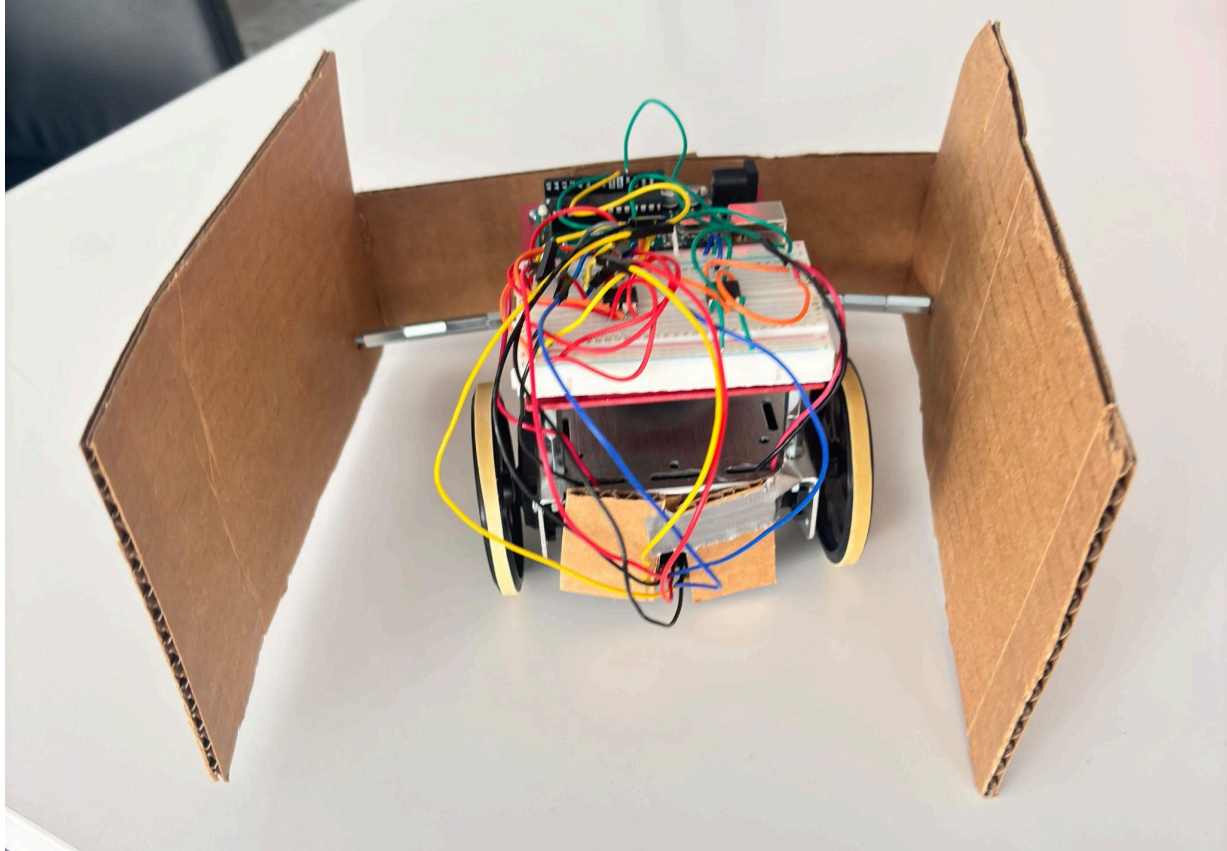
Group Number 45	Submission Date May 13, 2026
---------------------------	--

Name	Net ID
Chinmay Mangalgi	cm2236
Marcus Pang	mpp73
Edison Chen	ec876

1. Robot Design and Strategy Overview

How does your robot work mechanically, electrically, and in software? What does it ideally do during the competition? Insert one photo of your robot here. Put all other figures/diagrams in the appendices and reference them.

Our robot features a front funnel and side walls designed to contain the blocks within its perimeter. Additionally, a front wall positioned near the wheels helps prevent the robot from flipping if blocks slide underneath. Mechanically, this setup has proven effective in the competition, minimizing errors. Electrically, we employ a TC3200 color sensor connected to an Arduino board, which also wires the wheels. Our strategy is to drive straight until the sensor detects a color change, then turn right and continue. The related code is included in the appendix.



2. Design Process Reflection

How did your group progress through design, prototyping, and testing? Did you hit milestones on time? Significant roadblocks? Major design pivots?

We made steady progress each week through the design, prototyping, and testing stages. At the first milestone, the robot still pulled to the left, despite being programmed to drive straight. We checked the voltage to each motor with a multimeter, and both motors received equal voltage. There was no rubbing or slipping, and we attempted to fix the issue with rubber bands over the wheels. Until the final milestone, we designed our own arms and a barrier to prevent blocks from getting stuck in the wheels. The 3D-printed barrier was positioned too far forward, causing blocks to collect outside the perimeter. Because of this and due to time constraints, we switched to a simple cardboard design to meet the milestone requirements.

3. Competition Analysis

How did the robot perform on competition day? What worked, what didn't? Anything unexpected? Strengths and weaknesses?

We finished the round-robin with only 3 wins and were eliminated before the single-elimination bracket. The robot was reliably operational across all seven matches, as it never failed to start or get stuck on the border.

Strengths: The passive funnel proved to be the most reliable subsystem. Since it had no actuators, there was no risk of failure during the match: any cube the robot contacted at the right angle would end up within the perimeter.

Weaknesses: The primary limitation was the wheels. Whenever our robot collided with an opponent, the wheels would fall off, severely diminishing our chances of winning. Moving forward, we should secure our wheels to the servo to prevent them from coming loose. Also, in our code, before the robot turned, it would go backwards. Since there was nothing to keep the cubes in place in the funnel, they would go back out onto the floor, and we would lose cubes. The cardboard was flimsy and would bend around the cubes when picking them up.

4. Conclusions

What would you change if you could do it again with the same constraints and budget? What advice would you give next year's students?

Start designing the arms earlier, leaving more time to iterate and build a sturdier structure than cardboard alone can provide. Although we could not diagnose why the robot pulled to the left, a straight-moving robot with wheels screwed on would have let us progress further. For future students, early planning and a strong foundation enable more focus on competitiveness. Prioritize simple and reliable designs.

A fast robot reaching the middle is crucial for collecting blocks. We learned during the competition that servos significantly increase the chances of winning. You can't use stronger motors to control the wheels, but you can use gear ratios to mimic a more powerful servo.

Appendix A: Bill of Materials (BOM)

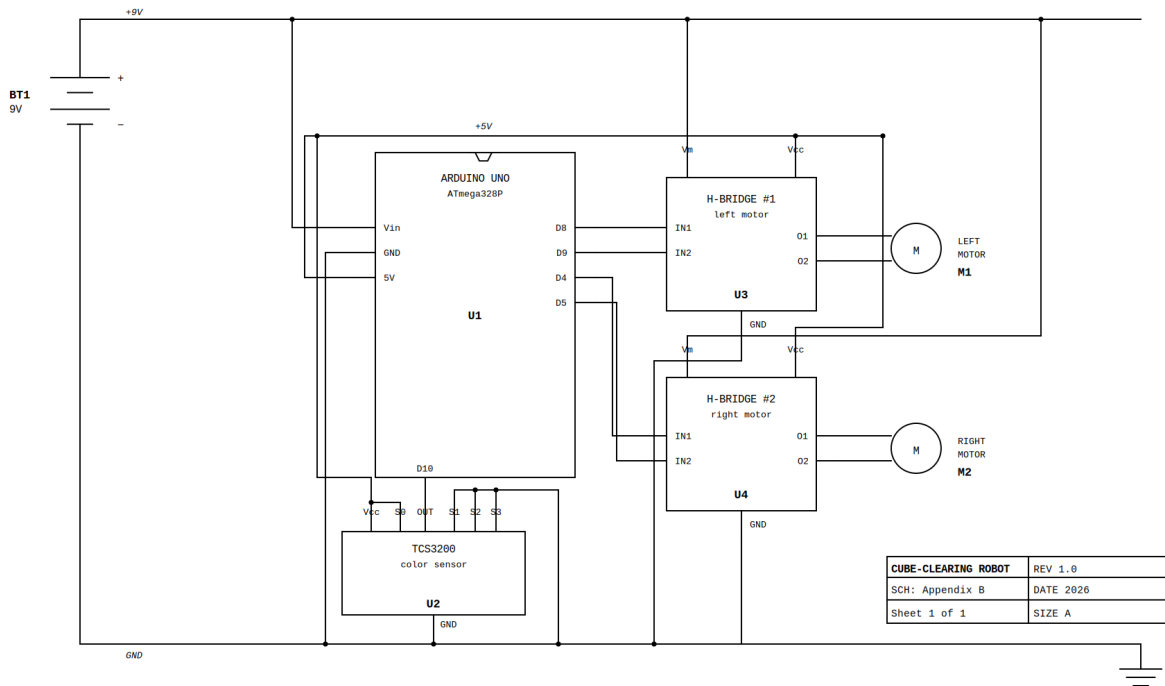
Include only parts your group purchased and counted toward your budget. Include parts that did not end up in the final design (3D prints, lasercut components, etc.).

#	Part / Description	Vendor	Qty	Unit Cost	Total	Used? (Y/N)
1	3D Printed Robot Arms	RPL	1	20.6	20.6	N
2						
3						
4						
5						
TOTAL				\$20.60	\$20.60	

We made one of the edges too thick and the beam protecting the wheels too far forward, and we quickly realized it wouldn't let us collect blocks. We scrapped this idea and moved to a box-funnel design that traps blocks within the walls as the robot moves.

Appendix B: Circuit Diagram

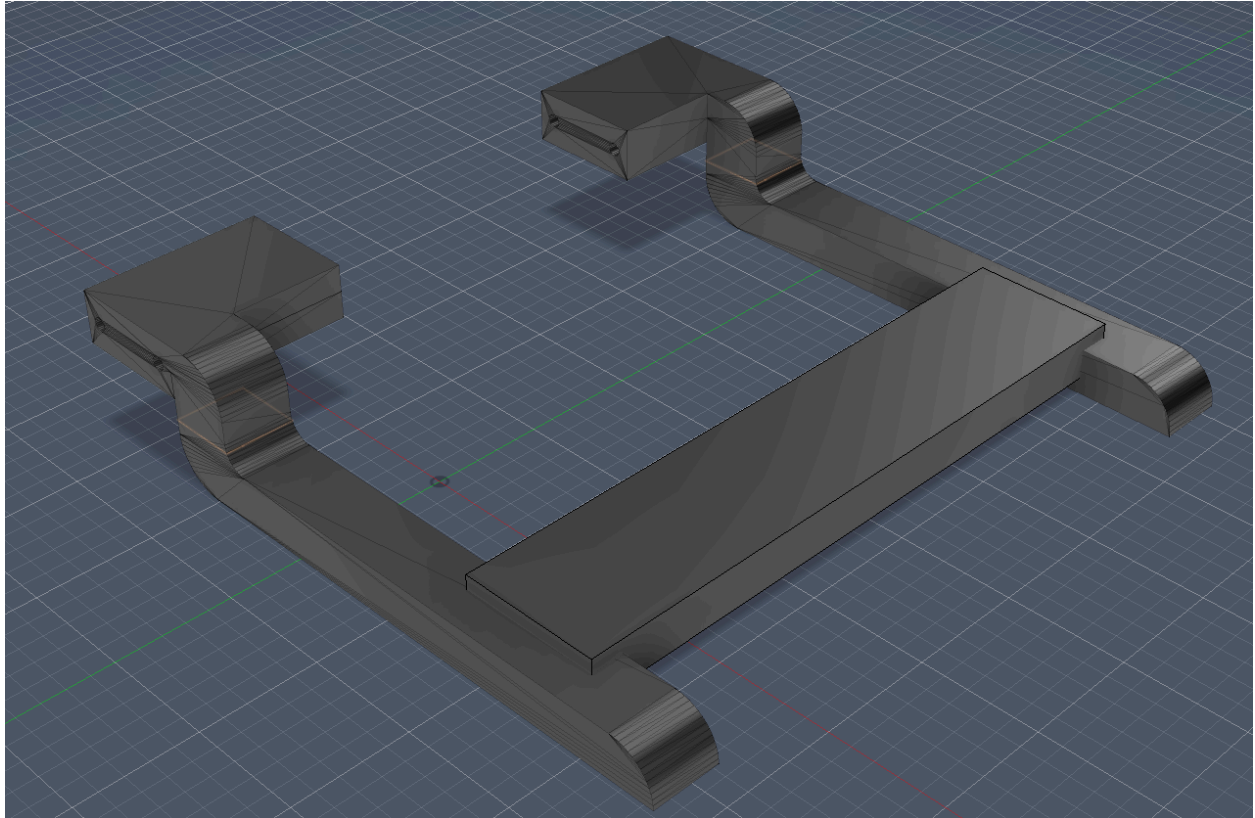
Use a schematic view (symbols for resistors, capacitors, etc.). Do NOT submit a Tinkercad "breadboard view" image — that earns a zero. A PowerPoint template with circuit symbols is on Canvas; any software is fine.



This diagram shows how our robot is wired. A 9V battery connects to the +9V rail. This powers both the Arduino's Vin and the motor supply (Vm) on each H-bridge. The Arduino's 5V output powers the color sensor and the logic side (Vcc) of both H-bridges. The first H-bridge drives the left motor, with inputs PB0/PB1 from D8/D9. On the other hand, the second H-bridge drives the right motor, with PD4/PD5 as its inputs from D4/D5. The sensor's OUT pin connects to D10 (PB2). S0 is tied to +5V and S1 to GND for 2% frequency scaling. S2 and S3 are tied to GND to lock the filter to the red channel.

Appendix C: CAD Files and Drawings

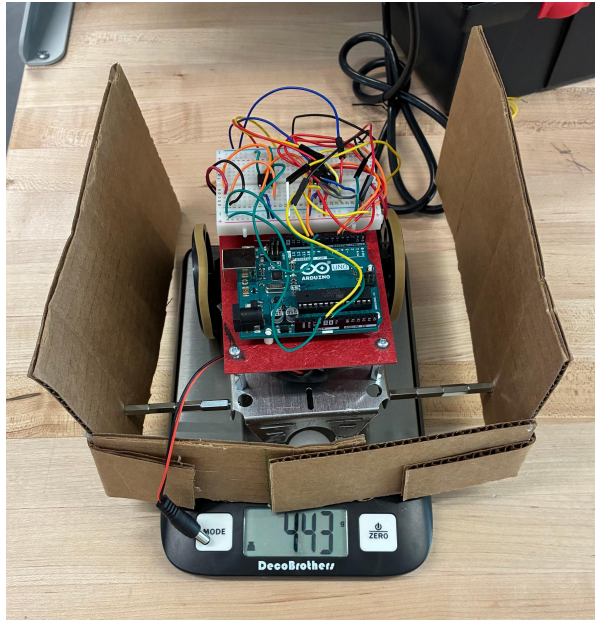
Include screenshots of relevant CAD files / drawings used for manufacturing parts. Also include screenshots of part volumes and weight calculations (or photos of parts being weighed) corresponding to the BOM.



This is the CAD file of the arms we 3D printed, but ended up not using.



3D printed robot arms being weighed. $49 \text{ times } .4 + 1 = \20.6

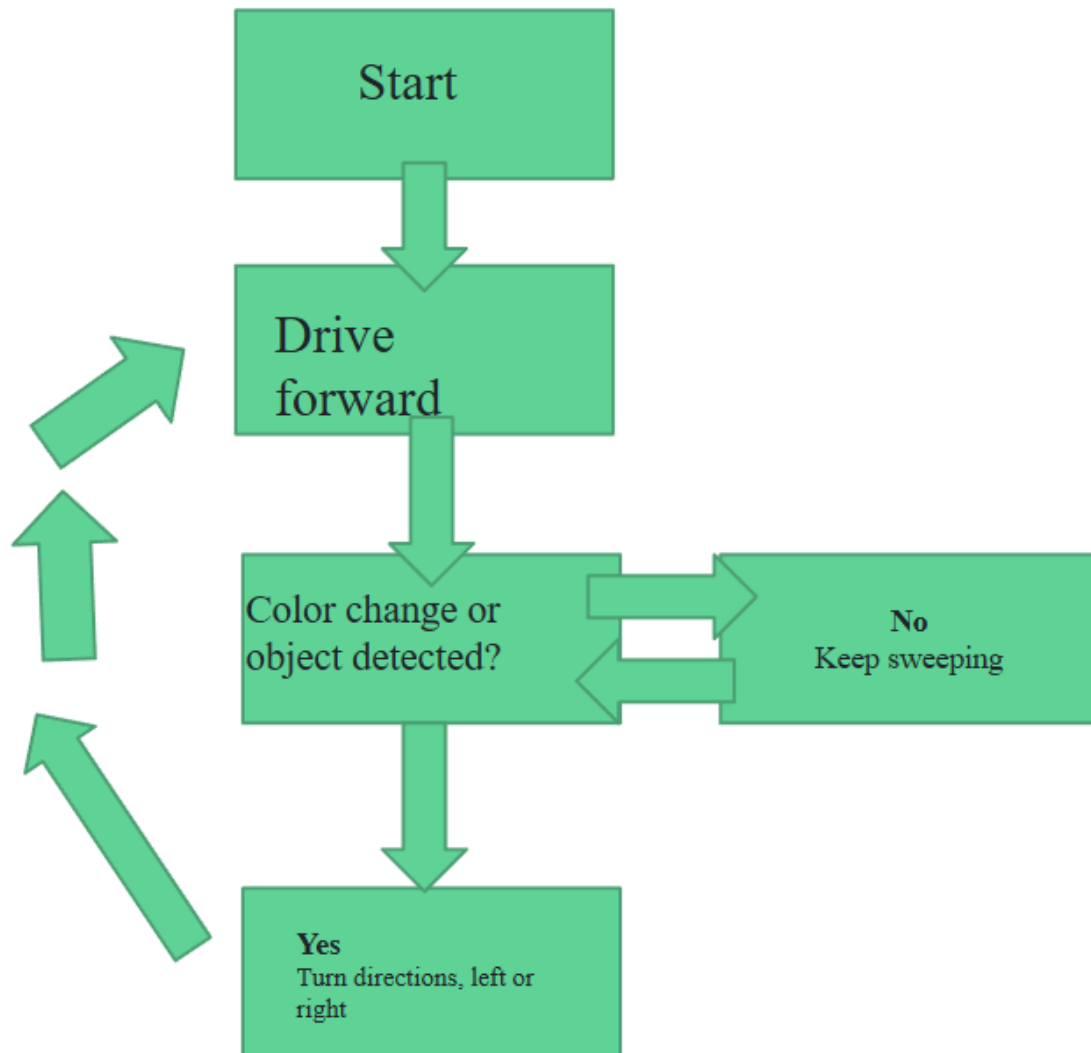


Images in Clockwise Order

- 1: Weight of the Robot in competition configuration (with cardboard arms attached)
- 2: weight of the individual arms + the chassis attachment mechanism
- 3, 4: pictures of just the cardboard arms + screws. Cardboard strips of various sizes were cut from spare boxes lying around the lab and assembled using hot glue, as shown in the image. Screw holes were poked in the cardboard by the screws. Washers (not shown) clamp the screws to the chassis.

Appendix D: Strategy Flowchart

Represent your robot's strategy / algorithm in flowchart form. Doesn't need every line of code — show high-level behaviors and functions.



Appendix E: Arduino Code

Commented Arduino code. A clear understanding of every line should be demonstrated. Reference any external sources you used.

```
#include <avr/io.h>
#include <util/delay.h>
#include <stdint.h>

// color sensor OUT pin -> D10 (PB2)
#define COLOR_PIN PB2

// pulse count thresholds for color ID
#define BLACK_THRESHOLD 20 // < 20 pulses = black
#define COLOR_THRESHOLD 100 // > 100 = yellow, in between = blue

// need 3 reads in a row before reacting (debounce)
#define COLOR_CONFIRM 3

// motor timings (ms)
#define SWITCH_PAUSE 300 // pause when flipping motor direction
#define DELAY_RIGHT_TURN 500 // ~90 deg right turn
#define BACKUP_TIME 300
#define BLIND_DRIVE 600 // drive past the line after turning so we don't
retrigger

// left motor: PB0/PB1, right motor: PD4/PD5
// motors are mounted mirrored so "LEFT_BACKWARD" is actually forward motion
#define LEFT_FORWARD() PORTB = (PORTB & 0b11111100) | 0b00000001
#define LEFT_BACKWARD() PORTB = (PORTB & 0b11111100) | 0b00000010
#define LEFT_STOP() PORTB = (PORTB & 0b11111100) | 0b00000000
#define RIGHT_FORWARD() PORTD = (PORTD & 0b11001111) | 0b00010000
#define RIGHT_BACKWARD() PORTD = (PORTD & 0b11001111) | 0b00100000
#define RIGHT_STOP() PORTD = (PORTD & 0b11001111) | 0b00000000

void stop_motors() {
    LEFT_STOP();
    RIGHT_STOP();
}
```

```

}

void drive_forward() {
    // stop first so the H-bridge doesn't shoot through
    LEFT_STOP();
    RIGHT_STOP();
    _delay_ms(SWITCH_PAUSE);
    LEFT_BACKWARD();
    RIGHT_FORWARD();
}

void drive_backward() {
    LEFT_STOP();
    RIGHT_STOP();
    _delay_ms(SWITCH_PAUSE);
    LEFT_FORWARD();
    RIGHT_BACKWARD();
}

void turn_right() {
    // spin in place
    LEFT_STOP();
    RIGHT_STOP();
    _delay_ms(SWITCH_PAUSE);
    LEFT_BACKWARD();
    RIGHT_BACKWARD();
    _delay_ms(DELAY_RIGHT_TURN);
    LEFT_STOP();
    RIGHT_STOP();
    _delay_ms(SWITCH_PAUSE);
}

// wait for one rising edge on the sensor pin, bail out after timeout iterations
uint8_t wait_rising_edge(uint32_t timeout) {
    uint32_t t = 0;
    // wait for line to go low first

```

```

    while (((PINB >> COLOR_PIN) & 1) && t++ < timeout);
    t = 0;
    // then wait for it to come back high
    while (!((PINB >> COLOR_PIN) & 1) && t++ < timeout);
    return t < timeout;
}

// count pulses over ~10ms (Timer1, /1024 prescale, 16MHz clock -> 64us/tick, 156
ticks)
uint16_t read_pulses() {
    uint16_t count = 0;
    TCCR1A = 0;
    TCCR1B = (1 << CS12) | (1 << CS10);
    TCNT1 = 0;
    while (TCNT1 < 156) {
        if (wait_rising_edge(5000)) count++;
    }
    TCCR1B = 0;
    return count;
}

#define COLOR_YELLOW 0
#define COLOR_BLUE 1
#define COLOR_BLACK 2

uint8_t read_color() {
    uint16_t c = read_pulses();
    if (c < BLACK_THRESHOLD) return COLOR_BLACK;
    if (c > COLOR_THRESHOLD) return COLOR_YELLOW;
    return COLOR_BLUE;
}

int main(void) {
    // motor pins as outputs (leave other bits alone)
    DDRB = (DDRB & 0b11111100) | 0b00000011;
    DDRD = (DDRD & 0b11001111) | 0b00110000;

```

```

// color sensor pin as input, no pullup (sensor drives it)
DDRB  &= ~(1 << COLOR_PIN);
PORTB &= ~(1 << COLOR_PIN);

_delay_ms(500); // let things settle on power-up

// figure out which zone we started in
// ignore black in case we start on/near a line
uint8_t start_color = read_color();
while (start_color == COLOR_BLACK) {
    start_color = read_color();
}

// drive forward, react to a black line or the opposite color, repeat forever
drive_forward();
uint8_t confirm = 0;
while (1) {
    uint8_t c = read_color();

    uint8_t change_detected = 0;
    if (c == COLOR_BLACK) change_detected = 1;
    if (start_color == COLOR_YELLOW && c == COLOR_BLUE)    change_detected = 1;
    if (start_color == COLOR_BLUE    && c == COLOR_YELLOW) change_detected = 1;

    if (change_detected) confirm++;
    else confirm = 0;

    if (confirm >= COLOR_CONFIRM) {
        stop_motors();
        drive_backward();
        _delay_ms(BACKUP_TIME);
        stop_motors();
        turn_right();
        drive_forward();
        _delay_ms(BLIND_DRIVE);
        confirm = 0;
    }
}

```

```
    }  
}  
  
return 0;  
}
```